

```
import heapq
import matplotlib.pyplot as plt
import numpy as np
```

```
ROWS, COLS = 3, 3
```

```
COLORS = {
    "empty": 0,
    "start": 1,
    "goal": 2,
    "visited": 3,
    "path": 4
}
```

```
color_map = [
    (1, 1, 1), # white
    (0, 1, 0), # green
    (1, 0, 0), # red
    (0, 0, 1), # blue
    (1, 1, 0) # yellow
]
```

```
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
def get_neighbors(node):
    moves = [(0,1), (1,0), (0,-1), (-1,0)]
    neighbors = []
    for m in moves:
        r, c = node[0] + m[0], node[1] + m[1]
```

```
    if 0 <= r < ROWS and 0 <= c < COLS:  
        neighbors.append((r, c))  
return neighbors
```

```
def show_grid(grid, g_score, goal):  
    plt.clf()  
    plt.imshow(grid, cmap=plt.matplotlib.colors.ListedColormap(color_map))  
    plt.grid(True)  
    plt.xticks(np.arange(-.5, COLS, 1))  
    plt.yticks(np.arange(-.5, ROWS, 1))  
    plt.gca().set_xticklabels([])  
    plt.gca().set_yticklabels([])
```

```
for i in range(ROWS):  
    for j in range(COLS):  
        node = (i, j)  
        if node in g_score:  
            g = g_score[node]  
            h = heuristic(node, goal)  
            f = g + h  
            text = f"f={f}\ng={g},h={h}"  
            plt.text(j, i, text, ha='center', va='center', fontsize=8)
```

```
plt.draw()  
plt.pause(0.8)
```

```
def astar(start, goal, grid):  
    open_set = []  
    heapq.heappush(open_set, (0, start))
```

```

came_from = {}
g_score = {start: 0}

step = 1

while open_set:
    _, current = heapq.heappop(open_set)

    print(f"\nStep {step}")
    print(f"Current Node: {current}")
    print(f"g={g_score[current]}, h={heuristic(current, goal)}, f={g_score[current] + heuristic(current, goal)}")

    if current != start and current != goal:
        grid[current] = COLORS["visited"]

    show_grid(grid, g_score, goal)

    if current == goal:
        print("\n🟢 Goal Reached! Reconstructing Path...")
        while current in came_from:
            current = came_from[current]
            if current != start:
                grid[current] = COLORS["path"]
                show_grid(grid, g_score, goal)
        return True

    for neighbor in get_neighbors(current):
        temp_g = g_score[current] + 1
        print(f"Checking Neighbor: {neighbor}")

```

```
if neighbor not in g_score or temp_g < g_score[neighbor]:  
    came_from[neighbor] = current  
    g_score[neighbor] = temp_g  
    f = temp_g + heuristic(neighbor, goal)  
  
    print(f'g={temp_g}, h={heuristic(neighbor, goal)}, f={f}')  
    heapq.heappush(open_set, (f, neighbor))
```

```
step += 1
```

```
return False
```

```
def main():
```

```
    grid = np.zeros((ROWS, COLS))
```

```
    print("Enter Start (row col) [0-2]: ")
```

```
    sr, sc = map(int, input().split())
```

```
    print("Enter Goal (row col) [0-2]: ")
```

```
    gr, gc = map(int, input().split())
```

```
    start = (sr, sc)
```

```
    goal = (gr, gc)
```

```
    grid[start] = COLORS["start"]
```

```
    grid[goal] = COLORS["goal"]
```

```
    plt.figure(figsize=(5,5))
```

```
    plt.ion()
```

```
found = astar(start, goal, grid)
```

```
plt.ioff()
```

```
plt.show()
```

```
if found:
```

```
    print("\n✅ Path Found Successfully!")
```

```
else:
```

```
    print("\n❌ No Path Found!")
```

```
if __name__ == "__main__":
```

```
    main()
```